Chapitre **18**

B-arbres

Les B-arbres sont des arbres de recherche équilibrés conçus pour être efficaces sur des disques magnétiques ou autres unités de stockage secondaires à accès direct. Les B-arbres ressemblent aux arbres rouge-noir (chapitre 13), mais ils sont plus performants quand il s'agit de minimiser les entrées-sorties disque.

La différence majeure entre les B-arbres et les arbres rouge-noir réside dans le fait que les nœuds des B-arbres peuvent avoir de nombreux enfant, jusqu'à plusieurs milliers. Autrement dit, le « facteur de ramification » d'un B-arbre peut être très grand, bien qu'il soit généralement déterminé par les caractéristiques de l'unité de disque utilisée. Les B-arbres ressemblent aux arbres rouge-noir au sens où tout B-arbre à n nœuds a une hauteur $O(\lg n)$, bien que la hauteur d'un B-arbre puisse être très inférieure à celle d'un arbre rouge-noir car son facteur de ramification peut être beaucoup plus grand. Cela permet donc également d'utiliser les B-arbres pour implémenter en temps $O(\lg n)$ de nombreuses opérations d'ensemble dynamique.

Les B-arbres généralisent de façon naturelle les arbres binaires de recherche. La figure 18.1 montre un B-arbre simple. Si un nœud interne x d'un B-arbre contient $n[x]$ clés, alors x possède $n[x] + 1$ enfants. Les clés du nœud x sont utilisées comme points de séparation de l'intervalle des clés gérées par x en $n[x] + 1$ sous-intervalles, chacun étant pris en charge par un enfant de x. Lorsqu'on recherche une clé dans un B-arbre, on prend une décision à $(n[x] + 1)$ alternatives, via des comparaisons avec les $n[x]$ clés stockées dans le nœud x. La structure des feuilles diffère de celle des nœuds internes ; nous étudierons ces différences à la section 18.1.

© Dunod – La photocopie non autorisée est un délit

Scanned with CamScanner

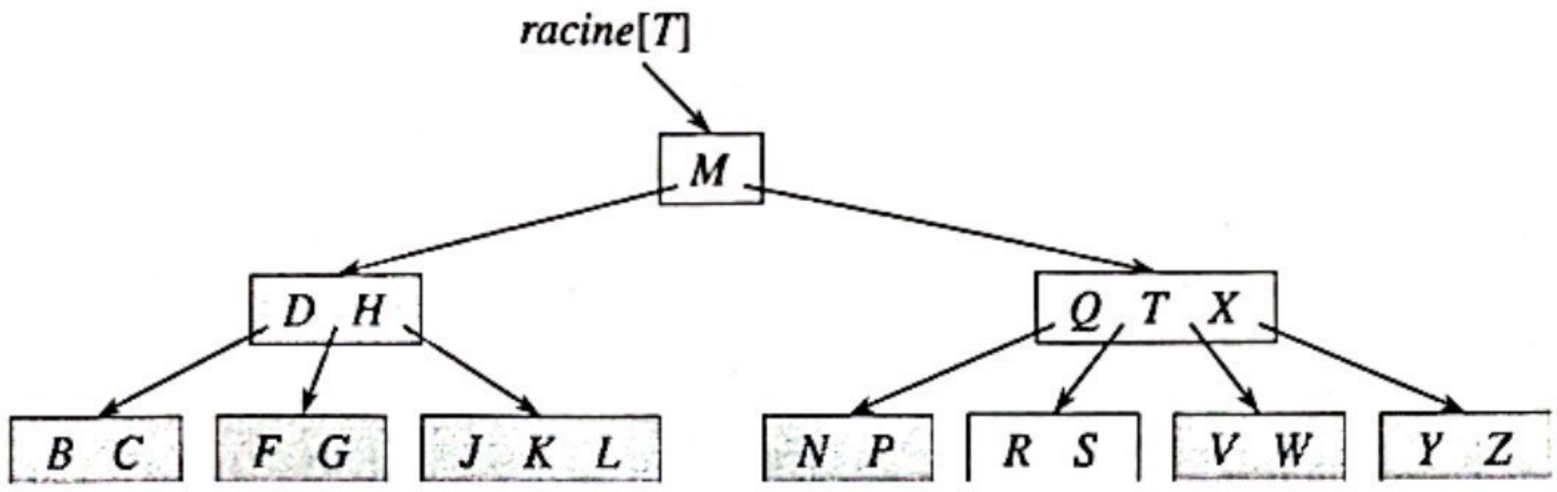


Figure 18.1 Un B-arbre dont les clés forment l'ensemble des consonnes. Un nœud interne x contenant $n[x]$ clés possède $n[x] + 1$ enfants. Toutes les feuilles ont la même profondeur dans l'arbre. Les nœuds parcourus pendant la recherche de la lettre R sont en gris clair.

La section 18.1 donne une définition précise des B-arbres et démontre que la hauteur d'un B-arbre croît seulement de manière logarithmique avec le nombre de nœuds qu'il contient. La section 18.2 décrit comment rechercher ou insérer une clé dans un B-arbre, et la section 18.3 traite de la suppression. Mais avant de continuer, il convient de se demander pourquoi les structures de données adaptées aux disques magnétiques demandent une évaluation différente de celles qui sont conçues pour fonctionner en mémoire principale.

a) Structures de données sur supports de stockage secondaires

Il existe de nombreuses techniques pour fournir de la mémoire à un système informatique. La ***mémoire centrale*** (ou ***mémoire principale***) d'un ordinateur est constituée habituellement de puces de silicone. Le coût de cette technologie (en termes de coût du bit stocké) est de l'ordre de cent fois le coût des stockages magnétiques comme les bandes ou les disques. Un système possède en général de la ***mémoire secondaire*** sous forme de disques magnétiques ; le volume de stockage de ces supports secondaires est souvent cent à mille fois plus important que la capacité de la mémoire principale.

La figure 18.2 montre un lecteur de disque classique. Le disque se compose de plusieurs ***plateaux***, qui tournent à vitesse constante autour d'un axe central commun. La surface de chaque plateau est recouverte de particules magnétisables. Chaque plateau est lu ou écrit par une ***tête*** située à l'extrémité d'un ***bras***. Les bras, qui sont reliés entre eux physiquement, peuvent rapprocher ou éloigner leurs têtes de l'axe central. Quand une tête est stationnaire, la surface qui défile au-dessous d'elle est une ***piste***. Les têtes de lecture-écriture sont alignées verticalement en permanence, et donc les pistes qui défilent sous elles sont accédées simultanément. La figure 18.2(b) montre un ensemble de pistes, connu sous le nom de ***cylindre***.

Les disques sont meilleur marché et ont plus de capacité que la mémoire centrale, mais ils sont beaucoup, beaucoup plus lents car ils renferment des composants mécaniques mobiles. Il y a deux types de mouvements mécaniques dans un disque : la rotation des plateaux et le déplacement des bras. À l'heure où nous écrivons ces lignes, les disques usuels tournent à des vitesses de 5 400 à 15 000 tours par minute,

Scanned with CamScanner

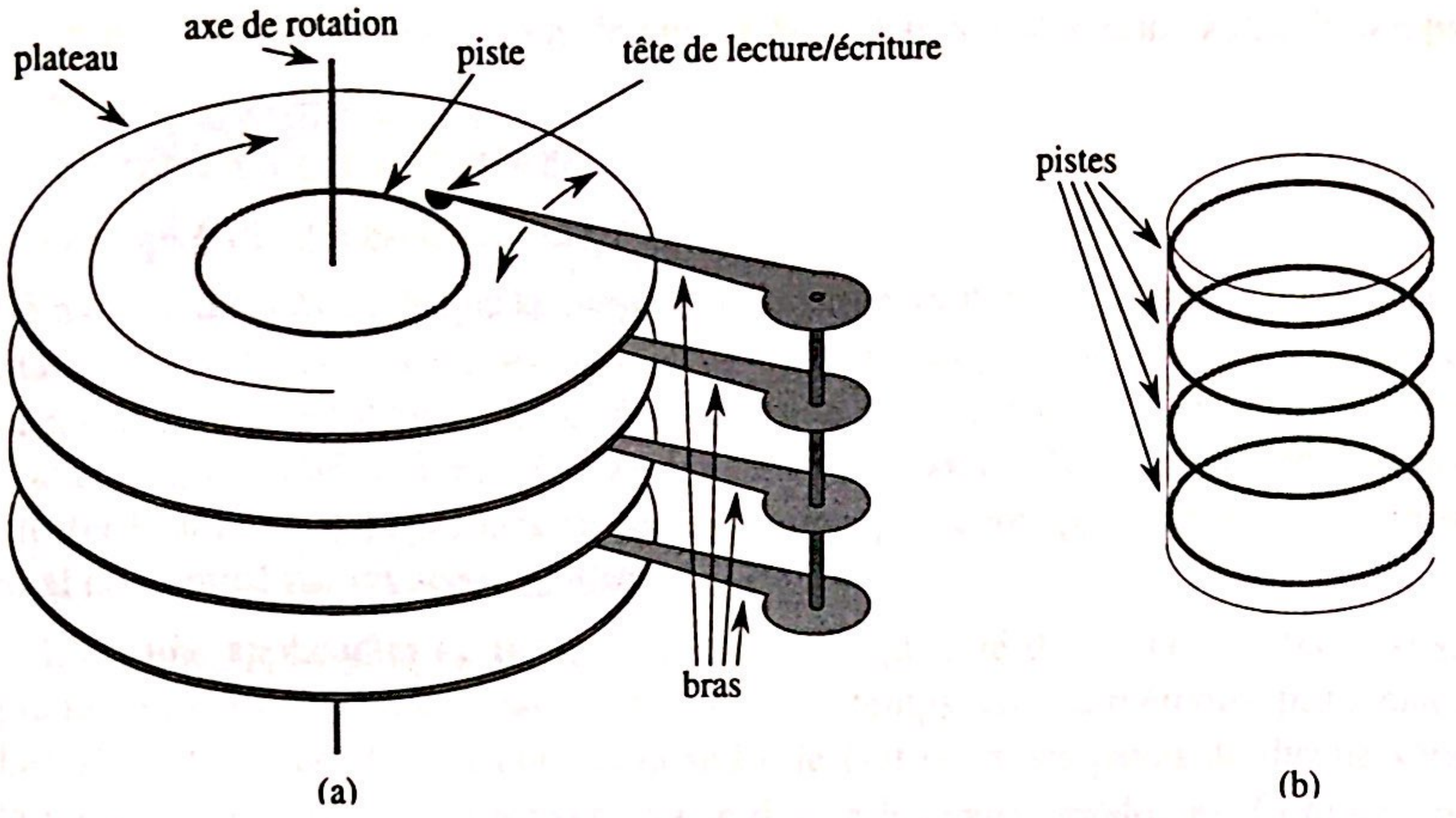


Figure 18.2 **(a)** Lecteur de disque typique. Il se compose de plusieurs plateaux tournant autour d'un axe central. La lecture et l'écriture de chaque plateau sont faites par une tête située à l'extrémité d'un bras. Les bras sont construits de façon telle que leurs têtes se déplacent à l'unisson. Ici, les bras tournent autour d'un axe pivot commun. Une piste est la surface qui défile sous la tête de lecture-écriture quand celle-ci est stationnaire. **(b)** Un cylindre se compose d'un ensemble de pistes coaxiales.

7 200 tr/mn étant la valeur la plus courante. 7 200 tr/mn peut sembler rapide ; mais un tour prend 8,33 ms, ce qui est presque cent mille fois plus lent que les temps d'accès de 100 nanosecondes qui est très courant pour les puces de silicone. En d'autres termes, si l'on doit attendre un tour complet pour qu'un certain élément vienne sous la tête de lecture-écriture, pendant ce délai on pourra accéder à la mémoire centrale 100 000 fois ! En moyenne on n'attend que le temps d'une demie révolution, mais la différence de temps d'accès entre puce de silicone et disque reste néanmoins énorme. Le déplacement des bras demande aussi du temps. À l'heure où nous écrivons ces lignes, le temps d'accès moyen pour un disque usuel est de l'ordre de 3 à 9 ms.

Pour amortir le temps passé à attendre les déplacements mécaniques, un disque accède à plusieurs éléments simultanément. Les données sont réparties en un certain nombre de ***pages*** de bits ayant la même taille et rangées de manière contiguë dans les cylindres ; chaque lecture ou écriture sur le disque implique une ou plusieurs pages toutes entières. Pour un disque typique, une page fait entre 2^{11} et 2^{14} octets de longueur. Une fois que la tête de lecture-écriture est positionnée correctement et que le disque a tourné pour que le début de la page désirée soit sous la tête, la lecture ou l'écriture est entièrement électronique (mis à part la rotation du disque), et il est alors possible de transférer rapidement de gros volumes de données.

Souvent, il est plus long pour l'ordinateur d'accéder à une page et de la lire sur un disque que de traiter ensuite toutes les données lues. C'est pour cette raison que, dans

© Dunod – La photocopie non autorisée est un délit

Scanned with CamScanner

ce chapitre, nous étudierons séparément les deux composantes principales du temps d'exécution :

- le nombre d'accès au disque et
- le temps CPU (temps de calcul).

Le nombre d'accès au disque se mesure en fonction du nombre de pages de données qui devront être lues ou écrites sur le disque. On remarque que le temps d'accès au disque n'est pas constant : il dépend de la distance entre la piste courante et la piste à atteindre, et aussi de l'état de rotation initial du disque. Nous allons néanmoins utiliser le nombre de pages lues ou écrites comme première approximation du temps total consommé par les accès au disque.

Dans une application de B-arbre classique, la quantité de données gérées est si grande qu'elles ne tiennent pas toutes en même temps dans la mémoire principale. Les algorithmes de B-arbre copient quand il le faut certaines pages du disque vers la mémoire principale et réécrivent sur le disque les pages modifiées. Comme ces algorithmes n'ont besoin que d'un nombre constant de pages en mémoire principale à un instant donné, la taille des B-arbres sur lesquels ils travaillent n'est pas limitée par celle de la mémoire principale.

Les opérations d'accès au disque sont modélisées dans notre pseudo code de la manière suivante. Soit x un pointeur vers un objet. Si l'objet se trouve dans la mémoire principale de l'ordinateur, on peut faire référence aux champs de l'objet de la façon habituelle : *clé*[x], par exemple. En revanche, si l'objet pointé par x se trouve sur le disque, on doit effectuer l'opération LIRE-DISQUE(x) pour amener l'objet x en mémoire principale avant de pouvoir référencer ses champs. (On suppose que, si x est déjà en mémoire principale, LIRE-DISQUE(x) n'exige pas d'accès disque ; c'est une non-opération ou « nop ».) De la même façon, l'opération ÉCRIRE-DISQUE(x) sert à sauvegarder toutes les modifications intervenues dans les champs de l'objet x. Autrement dit, le modèle général de travail sur un objet est le suivant :

```
1  x ← un pointeur vers un certain objet
2  LIRE-DISQUE(x)
3  opérations qui accèdent aux champs de x et/ou les modifient
4  ÉCRIRE-DISQUE(x)        ▷ Omis si aucun champ de x n'a été modifié.
5  autres opérations, qui accèdent aux champs de x sans les modifier
```

À un instant donné, le système ne peut garder qu'un nombre de pages limité en mémoire principale. On suppose que les pages qui ne sont plus utilisées sont vidées de la mémoire principale par le système ; nos algorithmes de B-arbre ignoreront ce problème.

Puisque dans la plupart des systèmes, le temps d'exécution d'un algorithme de B-arbre est surtout déterminé par le nombre d'opérations LIRE-DISQUE et ÉCRIRE-DISQUE qu'il effectue, il est logique d'utiliser au mieux ces opérations pour qu'elles lisent ou écrivent le plus d'informations possible à chaque accès. Un nœud de B-arbre

Scanned with CamScanner

aura donc le plus souvent la taille d'une page de disque entière. Le nombre d'enfants que pourra posséder un nœud est donc limité par la taille d'une page sur le disque.

Pour un grand B-arbre stocké sur un disque, on utilise fréquemment des facteurs de ramification compris entre 50 et 2000, qui dépendent du rapport entre la taille d'une clé et la taille d'une page. Un grand facteur de ramification réduit énormément à la fois la hauteur de l'arbre et le nombre d'accès disque nécessaires pour trouver une clé. La figure 18.3 montre un B-arbre ayant un facteur de ramification 1001 et une hauteur 2, capable de stocker plus d'un milliard de clés ; pourtant, comme le nœud situé à la racine peut être gardé en permanence dans la mémoire principale, il suffit de *deux* accès au disque pour accéder à une clé quelconque de cet arbre !

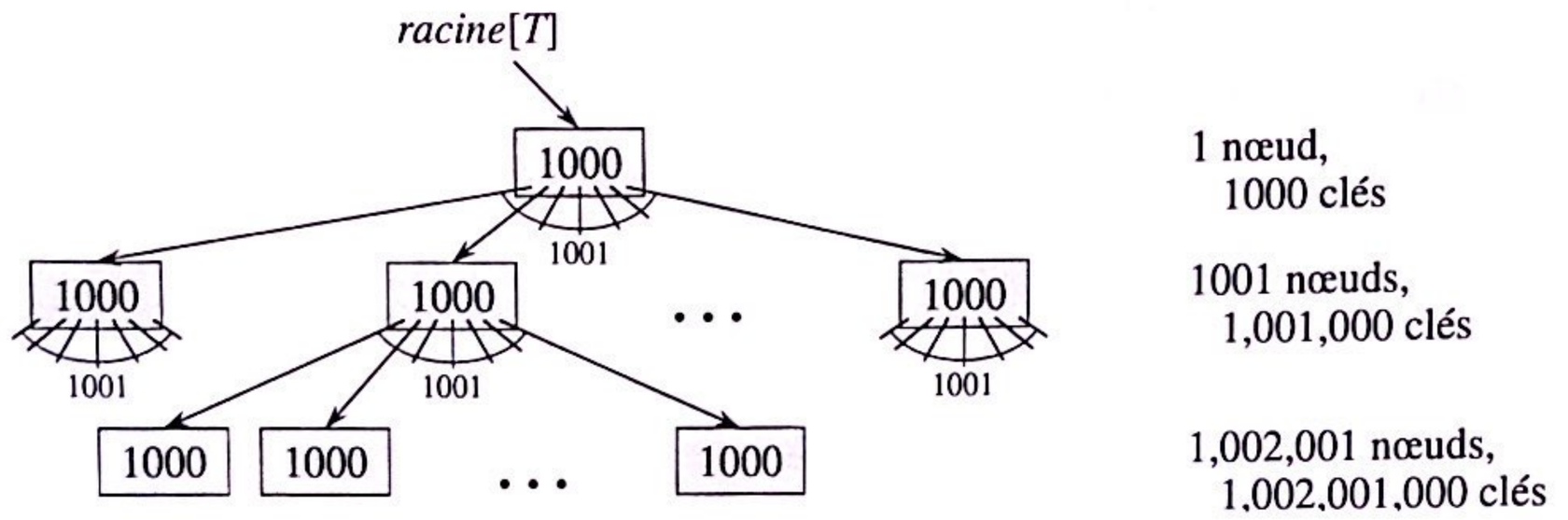


Figure 18.3 Un B-arbre de hauteur 2 contenant plus d'un milliard de clés. Chaque nœud (feuilles comprises) contient 1 000 clés. Il existe 1 001 nœuds à la profondeur 1 et plus d'un millions de feuilles à la profondeur 2. On peut voir à l'intérieur de chaque nœud la valeur $n[x]$, qui représente le nombre de clés de x.

18.1 DÉFINITION D'UN B-ARBRE

Pour simplifier on supposera, comme nous l'avons fait pour les arbres de recherche binaires et les arbres rouge-noir, que toute « donnée satellite » associée à une clé est stockée dans le même nœud que la clé. En pratique, il est possible de ne stocker avec chaque clé qu'un pointeur vers une autre page du disque, qui contiendra les informations satellites associées à cette clé. Le pseudo code de ce chapitre suppose implicitement que les informations satellites (ou le pointeur qui les adresse) voyagent avec la clé à chaque fois que la clé passe d'un nœud à l'autre. Une autre organisation fréquemment utilisée pour les B-arbres consiste à stocker toutes les informations satellites dans les feuilles et à ne conserver dans les nœuds internes que les clés et les pointeurs d'enfant, ce qui permet de maximiser le facteur de ramification des nœuds internes.

Un ***B-arbre*** T est une arborescence (de racine $racine[T]$) possédant les propriétés suivantes.

1) Chaque nœud x contient les champs ci-dessous :

   a) $n[x]$, le nombre de clés conservées actuellement par le nœud x,

© Dunod – La photocopie non autorisée est un délit

Scanned with CamScanner

b) les $n[x]$ clés elles-mêmes, stockées par ordre non décroissant :

$$clé_1[x] \leqslant clé_2[x] \leqslant \cdots \leqslant clé_{n[x]}[x],$$

c) *feuille*$[x]$, une valeur booléenne qui vaut VRAI si x est une feuille et FAUX si x est un nœud interne.

2) Chaque nœud interne x contient également $n[x] + 1$ pointeurs $c_1[x], c_2[x], \ldots, c_{n[x]+1}[x]$ vers ses enfant. Les feuilles n'ont pas d'enfants, et leurs champs c_i ne sont donc pas définis.

3) Les clés $clé_i[x]$ déterminent les intervalles de clés stockés dans chaque sous-arbre : si k_i est une clé stockée dans le sous-arbre de racine $c_i[x]$, alors

$$k_1 \leqslant clé_1[x] \leqslant k_2 \leqslant clé_2[x] \leqslant \cdots \leqslant clé_{n[x]}[x] \leqslant k_{n[x]+1} .$$

4) Toutes les feuilles ont la même profondeur, qui est la hauteur h de l'arbre.

5) Il existe un majorant et un minorant pour le nombre de clés pouvant être contenues par un nœud. Ces bornes peuvent être exprimées en fonction d'un entier fixé $t \geqslant 2$, appelé le ***degré minimal*** du B-arbre :

a) Tout nœud autre que la racine doit contenir au moins $t - 1$ clés. Tout nœud interne autre que la racine possède donc au moins t enfant. Si l'arbre n'est pas vide, la racine doit posséder au moins une clé.

b) Tout nœud peut contenir au plus $2t - 1$ clés. Un nœud interne peut donc posséder au plus $2t$ enfants. On dit qu'un nœud est ***complet*** s'il contient exactement $2t - 1$ clés. [1]



Le B-arbre le plus simple qui puisse être est celui pour lequel $t = 2$. Tout nœud interne possède alors 2, 3 ou 4 enfants et on se trouve en présence d'un ***arbre 2-3-4***. Toutefois, en pratique, on utilise des valeurs de t beaucoup plus grandes.

b) Hauteur d'un B-arbre

Le nombre d'accès disque nécessaires pour la plupart des opérations sur un B-arbre est proportionnel à la hauteur du B-arbre. Analysons à présent la hauteur d'un B-arbre dans le cas le plus défavorable.

Théorème 18.1 *Si $n \geqslant 1$, alors, pour tout B-arbre T à n clés de hauteur h et de degré minimal $t \geqslant 2$,*

$$h \leqslant \log_t \frac{n+1}{2} .$$

Démonstration : Si un B-arbre a une hauteur h, la racine contient au moins une clé et tous les autres nœuds contiennent au moins $t - 1$ clés. Par conséquent, il y a au moins 2 nœuds à la profondeur 1, au moins $2t$ nœuds à la profondeur 2, au moins $2t^2$

(1) Une autre variante courante de B-arbre, appelée ***B*-arbre***, exige que chaque nœud interne soit au moins rempli aux 2/3, et non pas rempli au moins à moitié comme l'exige un B-arbre.

Scanned with CamScanner

nœuds à la profondeur 3, etc. jusqu'à la profondeur h il y a au moins $2t^{h-1}$ nœuds. La figure 18.4 montre un tel arbre pour $h = 3$. Ainsi, le nombre n de clés obéit à l'inégalité

$$n \geqslant 1 + (t-1)\sum_{i=1}^{h} 2t^{i-1} = 1 + 2(t-1)\left(\frac{t^h - 1}{t-1}\right) = 2t^h - 1 .$$

Avec un peu d'algèbre élémentaire, l'on obtient $t^h \leqslant (n+1)/2$. En prenant les logarithmes de base t des deux membres, on démontre le théorème. ❑

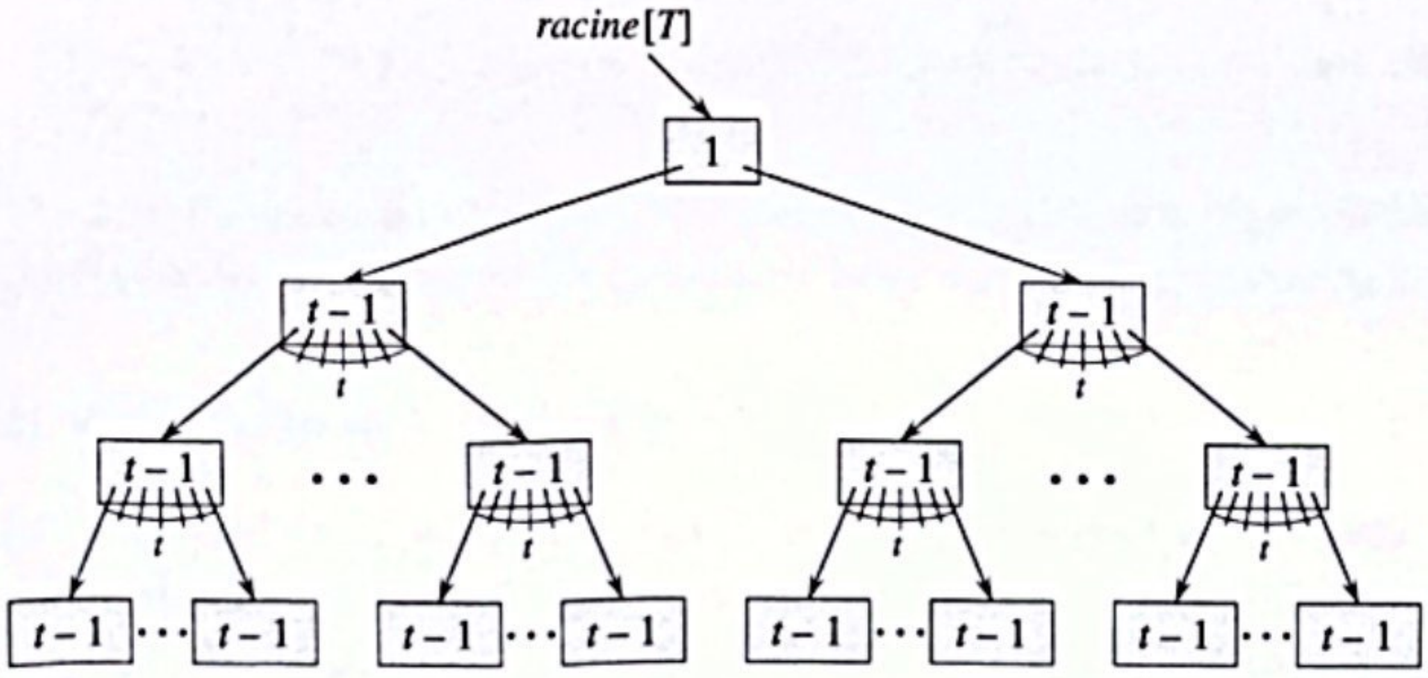


Figure 18.4 B-arbre de hauteur 3 contenant un nombre de clés minimal. À l'intérieur de chaque nœud x est affiché $n[x]$.

On voit ici la puissance des B-arbres comparés aux arbres rouge-noir. Bien que la hauteur de l'arbre augmente en $O(\lg n)$ dans les deux cas (n'oubliez pas que t est une constante), dans le cas des B-arbres la base du logarithme peut être très supérieure. Du coup, les B-arbres économisent un facteur d'environ $\lg t$ par rapport aux arbres rouge-noir, au niveau du nombre de nœuds examinés par la plupart des opérations d'arbre. Comme l'examen d'un nœud demande habituellement un accès disque, le nombre d'accès au disque est sensiblement réduit.

Exercices

18.1.1 Pourquoi n'autorise-t-on pas $t = 1$ comme degré minimal ?

18.1.2 Pour quelles valeurs de t l'arbre de la figure 18.1 est-il un B-arbre valide ?

18.1.3 Donner tous les B-arbres valides de degré minimal 2 permettant de représenter $\{1, 2, 3, 4, 5\}$.

18.1.4 En tant que fonction du degré minimal t, quel est le nombre maximal de clés qui peuvent être stockées dans un B-arbre de hauteur h ?

18.1.5 Décrire la structure de données que l'on obtiendrait si chaque nœud noir d'un arbre rouge-noir devait absorber ses enfants rouges, en incorporant leurs enfants aux siens.

© Dunod – La photocopie non autorisée est un délit

Scanned with CamScanner

18.2 OPÉRATIONS FONDAMENTALES SUR LES B-ARBRES

Dans cette section, nous présentons en détail les opérations RECHERCHER-B-ARBRE, CRÉER-B-ARBRE et INSÉRER-B-ARBRE. Pour ces procédures, nous adopterons deux conventions :

- La racine du B-arbre se trouve toujours en mémoire principale, de manière qu'une opération LIRE-DISQUE ne soit jamais nécessaire pour la racine ; cependant, un appel à ÉCRIRE-DISQUE sera nécessaire à chaque modification du nœud racine.
- Tout nœud passé en paramètre devra avoir subi auparavant une opération LIRE-DISQUE.

Les procédures présentées ici implémentent toutes des algorithmes « une passe » qui partent de la racine pour descendre l'arbre mais qui ne reviennent jamais en arrière.

a) Recherche dans un B-arbre

Les recherches dans un B-arbre ressemblent beaucoup à celles effectuées dans un arbre binaire de recherche, excepté qu'au lieu de prendre à chaque nœud une décision de branchement binaire (deux options au choix), on prend une décision à options multiples, selon le nombre d'enfants du nœud. Plus précisément, à chaque nœud interne x, on prend une décision parmi $(n[x] + 1)$ options possibles.

RECHERCHER-B-ARBRE est une généralisation directe de la procédure RECHERCHER-ARBRE définie pour les arbres binaires de recherche. RECHERCHER-B-ARBRE prend en entrée un pointeur vers la racine x d'un sous-arbre et une clé k à rechercher dans ce sous-arbre. L'appel de niveau supérieur est donc de la forme RECHERCHER-B-ARBRE($racine[T], k$). Si k se trouve dans le B-arbre, RECHERCHER-B-ARBRE retourne le couple (y, i) constitué d'un nœud y et d'un indice i tel que $clé_i[y] = k$. Sinon, la valeur NIL est retournée.

```
RECHERCHER-B-ARBRE(x, k)
1  i ← 1
2  tant que i ⩽ n[x] et k > clé_i[x]
3       faire i ← i + 1
4  si i ⩽ n[x] et k = clé_i[x]
5       alors retourner (x, i)
6  si feuille[x]
7       alors retourner NIL
8       sinon LIRE-DISQUE(c_i[x])
9             retourner RECHERCHER-B-ARBRE(c_i[x], k)
```

En employant une méthode de recherche linéaire, les lignes 1–3 trouvent le plus petit indice i pour lequel $k \leqslant clé_i[x]$ ou, s'il n'est pas trouvé, donnent à i la valeur $n[x] + 1$. Les lignes 4–5 testent si la clé a été découverte, auquel cas elle est retournée. Les lignes 6–9 permettent soit d'arrêter la recherche si elle échoue (si x est une feuille),

Scanned with CamScanner

soit d'effectuer une recherche récursive dans le sous-arbre de x approprié, après avoir effectué sur ce enfant le LIRE-DISQUE obligatoire.

La figure 18.1 illustre l'action de RECHERCHER-B-ARBRE ; les nœuds en gris clair sont ceux examinés pendant la recherche de la clé R.

Comme pour la procédure RECHERCHER-ARBRE des arbres binaires de recherche, les nœuds rencontrés pendant la récursivité forment un chemin qui descend de la racine de l'arbre. Le nombre de pages disque auxquelles RECHERCHER-B-ARBRE accède est donc $O(h) = O(\log_t n)$, où h est la hauteur du B-arbre et n le nombre de clés qu'il contient. Comme $n[x] < 2t$, le temps pris par la boucle **tant que** des lignes 2–3 à l'intérieur de chaque nœud est $O(t)$ et le temps CPU total est $O(th) = O(t \log_t n)$.

b) Création d'un B-arbre vide

Pour construire un B-arbre T, on commence par appeler CRÉER-B-ARBRE pour créer un nœud racine vide ; ensuite, on appelle INSÉRER-B-ARBRE pour ajouter de nouvelles clés. Ces deux procédures utilisent une procédure auxiliaire ALLOUER-NŒUD, qui alloue en $O(1)$ une page disque pour le nouveau nœud. On peut supposer qu'un nœud créé par ALLOUER-NŒUD ne nécessite aucun appel à LIRE-DISQUE, puisqu'il n'existe pour l'instant aucune information utile sur le disque concernant ce nœud.

```
CRÉER-B-ARBRE(T)
1  x ← ALLOUER-NŒUD()
2  feuille[x] ← VRAI
3  n[x] ← 0
4  ÉCRIRE-DISQUE(x)
5  racine[T] ← x
```

CRÉER-B-ARBRE requiert $O(1)$ opérations de disque, pour un temps CPU $O(1)$.

c) Insertion d'une clé dans un B-arbre

L'insertion d'une clé dans un B-arbre est nettement plus complexe que l'insertion d'une clé dans un arbre binaire de recherche. Comme c'est le cas avec un arbre binaire de recherche, on cherche la position de feuille à laquelle il faut insérer la nouvelle clé. Mais avec un B-arbre, on ne peut pas se contenter de créer une nouvelle feuille puis de l'insérer, car l'arbre résultant ne serait pas un B-arbre licite. À la place, on insère la nouvelle clé dans un nœud feuille existant. Comme on ne peut pas insérer une clé dans un nœud feuille qui est plein, on introduit une opération qui ***partage*** un nœud y plein (ayant $2t - 1$ clés) autour de sa ***clé médiane*** $clé_t[y]$, pour en faire deux nœuds ayant chacun $t - 1$ clés. La clé médiane remonte dans le parent de y pour identifier le point de partage entre les deux nouveaux arbres. Mais si le parent de y est plein, lui aussi, il faut le partager avant d'y insérer la nouvelle clé ; par conséquent, il se peut que ce processus de partage de nœuds pleins se propage dans tout l'arbre (en partant du bas pour aller vers le haut).

© Dunod – La photocopie non autorisée est un délit

Scanned with CamScanner

Comme c'est le cas avec un arbre binaire de recherche, on peut insérer une clé dans un B-arbre en une seule passe qui descend l'arbre depuis la racine pour arriver sur une feuille. Pour ce faire, on n'attend pas de savoir s'il faudra partager un nœud plein pour faire l'insertion. À la place, à mesure que l'on descend l'arbre pour chercher la position de la nouvelle clé, on partage chaque nœud plein que l'on rencontre sur le chemin (y compris la feuille elle-même). Ainsi, quand on voudra partager un nœud plein y, on sera assuré que son parent n'est pas plein.

➤ Partage de nœud dans un B-arbre

La procédure PARTAGER-ENFANT-B-ARBRE prend en entrée un nœud interne *non plein* x (supposé être en mémoire centrale), un indice i et un nœud y (censé, lui aussi, être en mémoire centrale), tels que $y = c_i[x]$ soit un enfant *plein* de x. La procédure partage alors cet enfant en deux et modifie x pour qu'il ait un enfant de plus. (Pour partager une racine pleine, on commence par faire de la racine un enfant d'un nouveau nœud racine vide de façon à pouvoir utiliser PARTAGER-ENFANT-B-ARBRE). La hauteur de l'arbre augmente donc de un ; le partage est la seule façon de faire croître l'arbre.

La figure 18.5 illustre ce processus. Le nœud plein y est partagé au niveau de sa clé médiane S, qui remonte vers le parent de y, à savoir le nœud x. Les clés de y qui sont supérieures à la clé médiane vont dans un nouveau nœud z, qui devient un nouvel enfant de x.

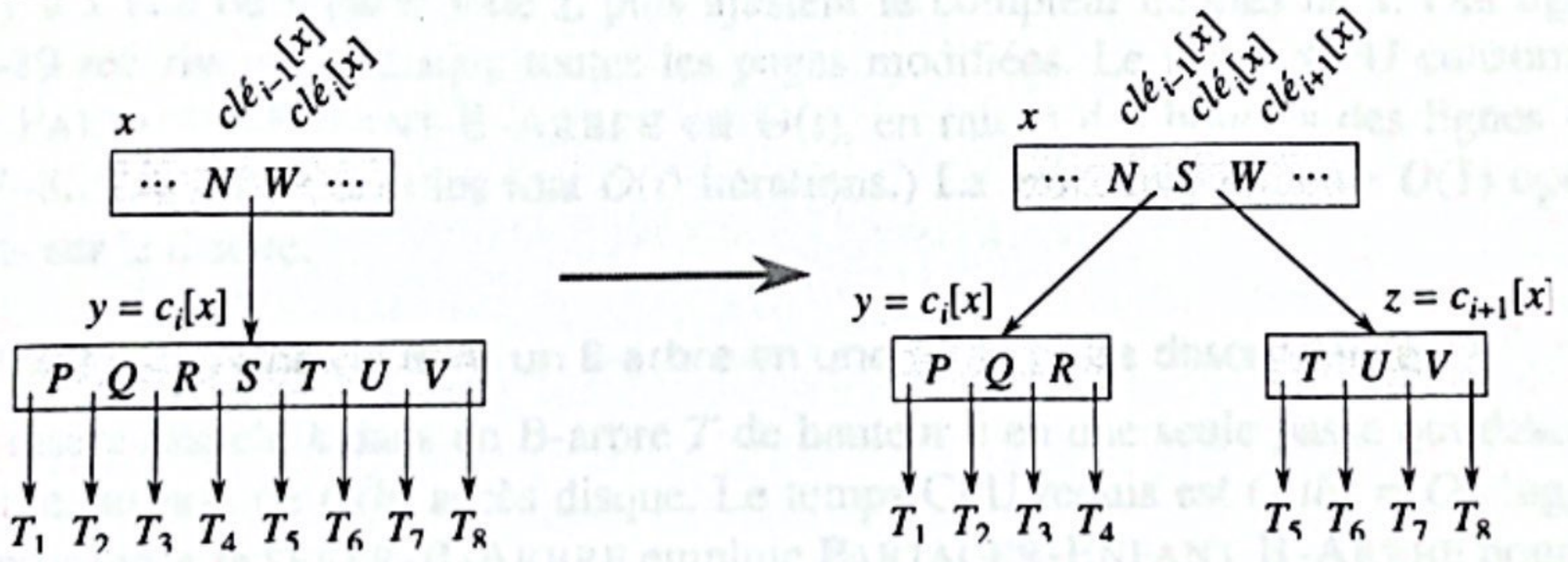


Figure 18.5 Partage d'un nœud avec $t = 4$. Le nœud y est partagé en deux nœuds, y et z, et la clé médiane S de y va dans le parent de y.

```
PARTAGER-ENFANT-B-ARBRE(x, i, y)
1  z ← ALLOUER-NŒUD()
2  feuille[z] ← feuille[y]      z se trouve au même niveau que y.
3  n[z] ← t − 1
4  pour j ← 1 à t − 1
5      faire clé_j[z] ← clé_{j+t}[y]      ← on lui transfère la moitié des clefs.
6  si non feuille[y]
7      alors pour j ← 1 à t
8                faire c_j[z] ← c_{j+t}[y]      ) on lui affecte les sous arbres
```

Scanned with CamScanner

```
 9  n[y] ← t − 1
10  pour j ← n[x] + 1 jusqu'à i + 1
11      faire c_{j+1}[x] ← c_j[x]
12  c_{i+1}[x] ← z
13  pour j ← n[x] jusqu'à i
14      faire clé_{j+1}[x] ← clé_j[x]
15  clé_i[x] ← clé_t[y]
16  n[x] ← n[x] + 1
17  ÉCRIRE-DISQUE(y)
18  ÉCRIRE-DISQUE(z)
19  ÉCRIRE-DISQUE(x)
```

on corrige les clés de a.

PARTAGER-ENFANT-B-ARBRE fonctionne en faisant du « couper-coller » direct. Ici, y est le $i$ème enfant de x et c'est le nœud qui est partagé. Le nœud y a, à l'origine, $2t$ enfants ($2t - 1$ clés), mais il est réduit à t enfants ($t - 1$ clés) par cette opération. Le nœud z « adopte » les t plus grands enfants ($t - 1$ clés) de y, et z devient un nouvel enfant de x, placé juste après y dans la table des enfants de x. La clé médiane de y remonte pour devenir la clé de x qui sépare y et z.

Les lignes 1–8 créent le nœud z, puis lui donnent les $t - 1$ plus grandes clés et les t enfants correspondants de y. La ligne 9 modifie le compteur de clés de y. Enfin, les lignes 10–16 insèrent z en tant qu'enfant de x, font remonter la clé médiane de y à x afin de séparer y de z, puis ajustent le compteur de clés de x. Les lignes 17–19 réécrivent sur disque toutes les pages modifiées. Le temps CPU consommé par PARTAGER-ENFANT-B-ARBRE est $\Theta(t)$, en raison des boucles des lignes 4–5 et 7–8. (Les autres boucles font $O(t)$ itérations.) La procédure effectue $O(1)$ opérations sur le disque.

➤ Insertion d'une clé dans un B-arbre en une seule passe descendante

On insère une clé k dans un B-arbre T de hauteur h en une seule passe qui descend l'arbre, au prix de $O(h)$ accès disque. Le temps CPU requis est $O(th) = O(t \log_t n)$. La procédure INSÉRER-B-ARBRE emploie PARTAGER-ENFANT-B-ARBRE pour assurer que la récursivité ne descendra jamais sur un nœud plein.

```
INSÉRER-B-ARBRE(T, k)
 1  r ← racine[T]
 2  si n[r] = 2t − 1
 3     alors s ← ALLOUER-NŒUD()
 4           racine[T] ← s
 5           feuille[s] ← FAUX
 6           n[s] ← 0
 7           c_1[s] ← r
 8           PARTAGER-ENFANT-B-ARBRE(s, 1, r)
 9           INSÉRER-B-ARBRE-INCOMPLET(s, k)
10     sinon INSÉRER-B-ARBRE-INCOMPLET(r, k)
```

© Dunod – La photocopie non autorisée est un délit

Scanned with CamScanner

Les lignes 3–9 gèrent le cas où le nœud racine r est plein : la racine est alors partagée, et un nouveau nœud s (ayant deux enfants) devient la racine. Partager la racine est l'unique façon d'augmenter la hauteur d'un B-arbre. La figure 18.6 illustre ce cas. Contrairement à un arbre binaire de recherche, un B-arbre croît par le haut et non par le bas. La procédure s'achève par l'appel de INSÉRER-B-ARBRE-INCOMPLET pour faire l'insertion de la clé k dans l'arbre enraciné sur le nœud racine non plein. INSÉRER-B-ARBRE-INCOMPLET fait autant d'appels récursifs que nécessaire au cours de sa descente de l'arbre, garantissant à tout instant que le nœud sur lequel elle fait de la récursivité n'est pas plein, et ce en appelant PARTAGER-ENFANT-B-ARBRE en fonction des besoins.

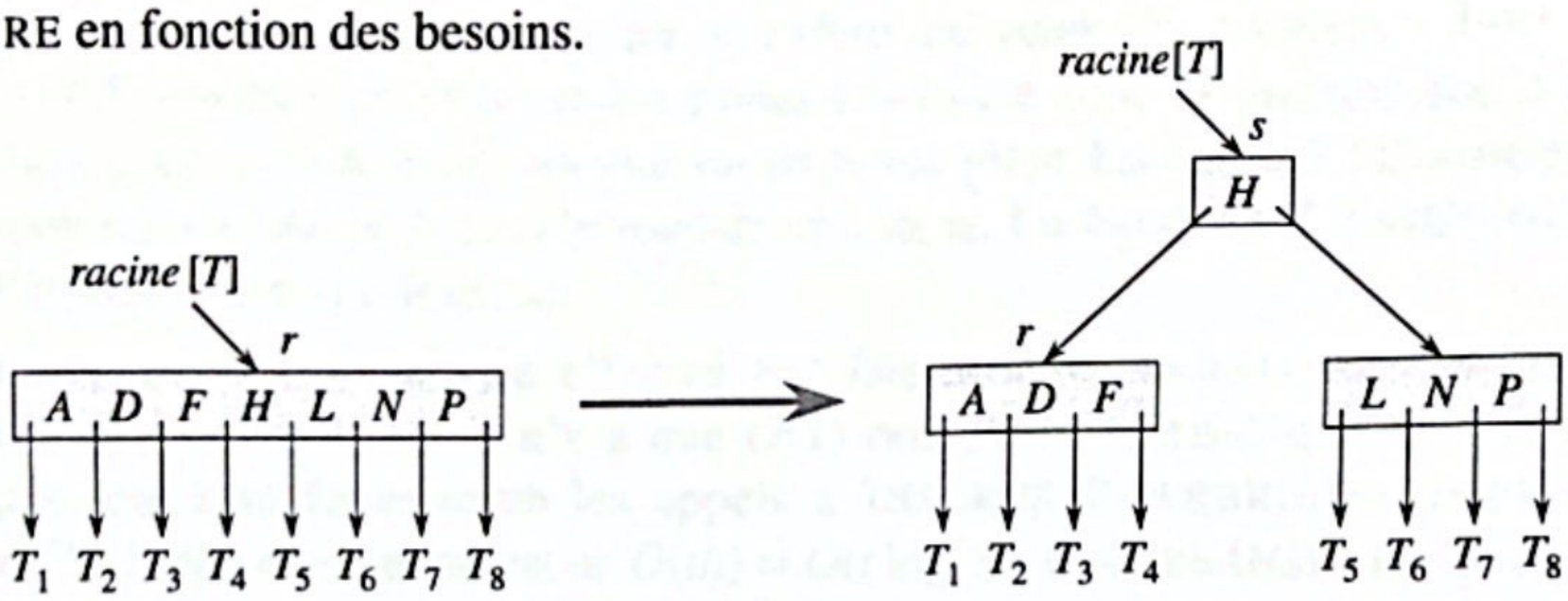


Figure 18.6 Partage de la racine avec $t = 4$. Le nœud racine r est partagé en deux, et il y a création d'un nouveau nœud racine s. La nouvelle racine contient la clé médiane de r et a comme enfants les deux moitiés de r. Le B-arbre croit d'une unité quand il y a partage de la racine.

La procédure auxiliaire récursive INSÉRER-B-ARBRE-INCOMPLET insère la clés k dans le nœud x, qui est censé être non plein lors de l'appel de la procédure. Le fonctionnement de INSÉRER-B-ARBRE et le fonctionnement récursif de INSÉRER-B-ARBRE-INCOMPLET assurent que cette hypothèse est vraie.

```
INSÉRER-B-ARBRE-INCOMPLET(x, k)
 1  i ← n[x]
 2  si feuille[x]
 3     alors tant que i ⩾ 1 et k < clé_i[x]
 4              faire clé_{i+1}[x] ← clé_i[x]
 5                   i ← i − 1
 6           clé_{i+1}[x] ← k
 7           n[x] ← n[x] + 1
 8           ÉCRIRE-DISQUE(x)
 9     sinon tant que i ⩾ 1 et k < clé_i[x]
10              faire i ← i − 1
11           i ← i + 1
12           LIRE-DISQUE(c_i[x])
13           si n[c_i[x]] = 2t − 1
14              alors PARTAGER-ENFANT-B-ARBRE(x, i, c_i[x])
15                    si k > clé_i[x]
16                       alors i ← i + 1
17           INSÉRER-B-ARBRE-INCOMPLET(c_i[x], k)
```

Scanned with CamScanner

La procédure INSÉRER-B-ARBRE-INCOMPLET fonctionne ainsi. Les lignes 3–8 gèrent le cas où x est une feuille, en insérant la clé k dans x. Si x n'est pas une feuille, alors on doit insérer k dans le nœud feuille idoine du sous-arbre enraciné sur le nœud interne x. Dans ce cas, les lignes 9–11 déterminent l'enfant de x sur lequel doit descendre la récursivité. La ligne 13 détecte si la récursivité descendrait sur un enfant plein, auquel cas la ligne 14 utilise PARTAGER-ENFANT-B-ARBRE pour diviser cet enfant en deux enfants non pleins, et les lignes 15–16 déterminent lequel des deux enfants est maintenant l'enfant sur lequel il faut descendre. (Notez que point n'est besoin d'un appel LIRE-DISQUE($c_i[x]$) après que la ligne 16 a incrémenté i, car la récursivité descendra dans ce cas sur un enfant qui vient d'être créé par PARTAGER-ENFANT-B-ARBRE.) L'effet net des lignes 13–16 est donc de garantir que la procédure ne va jamais faire de récursivité sur un nœud plein. La ligne 17 fait ensuite de la récursivité pour insérer k dans le sous-arbre idoine. La figure 18.7 illustre les divers cas d'insertion dans un B-arbre.

Le nombre d'accès disque effectué par INSÉRER-B-ARBRE est $O(h)$ pour un B-arbre de hauteur h, car il n'y a que $O(1)$ opérations LIRE-DISQUE et ÉCRIRE-DISQUE qui sont faites entre les appels à INSÉRER-B-ARBRE-INCOMPLET. Le temps CPU total consommé est de $O(th) = O(t \log_t n)$. Comme INSÉRER-B-ARBRE-INCOMPLET est récursive par la queue, on peut aussi la mettre en œuvre sous la forme d'une boucle **tant que**, démontrant que le nombre de pages qui doivent résider en mémoire centrale à un instant quelconque est $O(1)$.

Exercices

18.2.1 Montrer les résultats de l'insertion successive des clés

$$F, S, Q, K, C, L, H, T, V, W, M, R, N, P, A, B, X, Y, D, Z, E$$

dans un B-arbre vide de degré minimal 2. Ne représenter que les configurations de l'arbre qui précèdent immédiatement le découpage d'un nœud, puis représenter la configuration finale.

18.2.2 Décrire dans quelles circonstances éventuelles l'exécution d'un appel INSÉRER-B-ARBRE pourrait engendrer des opérations LIRE-DISQUE ou ÉCRIRE-DISQUE redondantes. (Un LIRE-DISQUE redondant concerne une page qui se trouve déjà en mémoire. Un ÉCRIRE-DISQUE redondant écrit sur le disque une page identique à celle déjà présente sur le disque.)

18.2.3 Dire comment trouver la clé minimale stockée dans un B-arbre et comment trouver le prédécesseur d'une clé donnée d'un B-arbre.

18.2.4 ⋆ On suppose que les clés $\{1, 2, \ldots, n\}$ sont insérées dans un B-arbre vide de degré minimal 2. Combien de nœuds possédera le B-arbre final ?

18.2.5 Comme les feuilles n'ont pas besoin de pointeurs d'enfant, on pourrait concevoir qu'elles utilisent une valeur de t différente (plus grande) que les nœuds internes pour la même taille de page disque. Montrer comment modifier les procédures de création et d'insertion dans un B-arbre pour gérer cette variante.

© Dunod – La photocopie non autorisée est un délit

Scanned with CamScanner

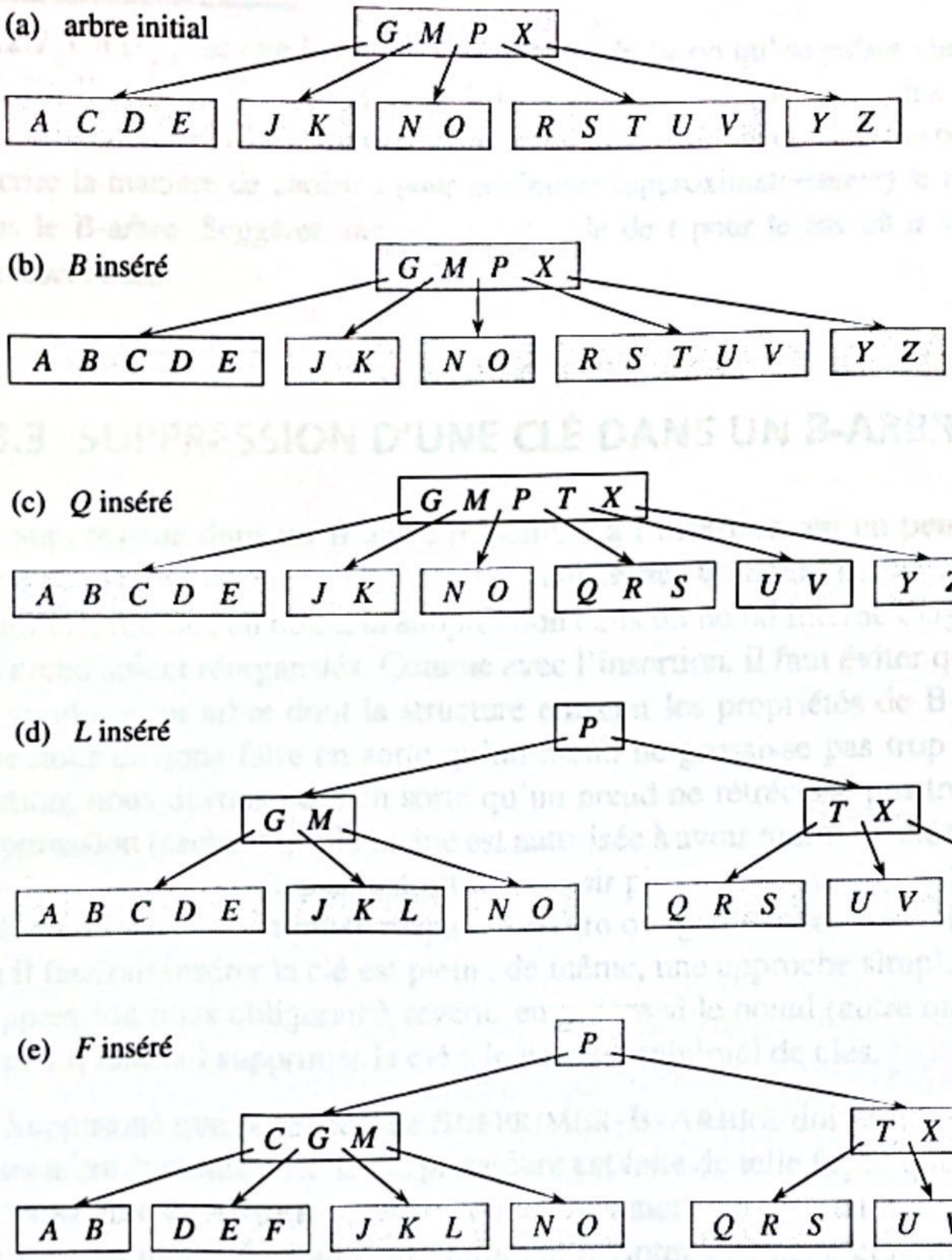


Figure 18.7 Insertion de clés dans un B-arbre. Le degré minimal t de ce B-arbre est 3, de sorte qu'un nœud peut contenir au plus 5 clés. Les nœuds en gris clair sont ceux qui seront modifiés par le processus d'insertion. **(a)** L'arbre initial de cet exemple. **(b)** Le résultat de l'insertion de *B* dans l'arbre initial ; c'est une insertion simple dans une feuille. **(c)** Le résultat de l'insertion de *Q* dans l'arbre précédent. Le nœud *RSTUV* est divisé en deux nœuds contenant *RS* et *UV*, la clé *T* remonte dans la racine et *Q* est inséré dans la moitié de gauche (nœud *RS*). **(d)** Le résultat de l'insertion de *L* dans l'arbre précédent. La racine est divisée immédiatement car elle est pleine, et le B-arbre croît en hauteur de un. Ensuite, *L* est inséré dans la feuille contenant *JK*. **(e)** Le résultat de l'insertion de *F* dans l'arbre précédent. Le nœud *ABCDE* est divisé, puis *F* est inséré dans la moitié de droite (nœud *DE*).

18.2.6 On suppose que RECHERCHER-B-ARBRE est implémentée de manière à utiliser une recherche dichotomique, plutôt qu'une recherche linéaire, à l'intérieur de chaque nœud. Montrer que le temps CPU requis devient alors $O(\lg n)$, indépendamment de la façon dont t est choisi comme fonction de n.

Scanned with CamScanner

18.2.7 On suppose que le disque est construit de façon qu'on puisse choisir arbitrairement la taille d'une page disque, mais que le temps pris pour lire la page soit $a + bt$, où a et b sont des constantes spécifiées et t le degré minimal d'un B-arbre utilisant des pages de cette taille. Décrire la manière de choisir t pour minimiser (approximativement) le temps de recherche dans le B-arbre. Suggérer une valeur optimale de t pour le cas où $a = 5$ ms et $b = 10$ microsecondes.

18.3 SUPPRESSION D'UNE CLÉ DANS UN B-ARBRE

La suppression dans un B-arbre ressemble à l'insertion, en un peu plus compliqué. En effet, la suppression d'une clé peut concerner un nœud quelconque et pas seulement une feuille ; en outre, la suppression dans un nœud interne exige que les enfants du nœud soient réorgansiés. Comme avec l'insertion, il faut éviter qu'une suppresion ne produise un arbre dont la structure enfreint les propriétés de B-arbre. De même que nous devions faire en sorte qu'un nœud ne grossisse pas trop pour cause d'insertion, nous devons faire en sorte qu'un nœud ne rétrécisse pas trop pour cause de suppression (sachant que la racine est autorisée à avoir moins de clés que le minimum $t - 1$, mais qu'elle n'a pas le droit d'en avoir plus que le maximum $2t - 1$). Un algorithme d'insertion simpliste risquerait d'être obligé de rebrousser chemin si le nœud où il faudrait insérer la clé est plein ; de même, une approche simpliste en matière de suppression nous obligerait à revenir en arrière si le nœud (autre que la racine) dans lequel il faudrait supprimer la clé a le nombre minimal de clés.

Supposons que la procédure SUPPRIMER-B-ARBRE doive supprimer la clé k du sous-arbre enraciné en x. Cette procédure est faite de telle façon que, chaque fois que SUPPRIMER-B-ARBRE est appelée récursivement sur un nœud x, le nombre de clés de x est au moins égal au degré minimal t. Notez que cette condition exige une clé de plus que le minimum requis par les conditions de B-arbre ; il peut donc advenir qu'une clé soit obligée d'aller dans un nœud enfant avant que la récursivité descende sur cet enfant. Cette condition renforcée permet de supprimer une clé de l'arbre en une seule passe descendante, sans risque de « retour en arrière » (à une seule exception, que nous allons expliquer). À propos des spécifications suivantes concernant la suppression dans un B-arbre, il faut bien comprendre ceci : s'il arrive que le nœud racine x devienne un nœud interne n'ayant pas de clés (cette situation peut se produire dans les cas 2c et 3b, donnés ci-après), alors x est supprimé et l'unique enfant de x, à savoir $c_1[x]$, devient la nouvelle racine de l'arbre (dont la hauteur diminue de un) et préserve la propriété énonçant que la racine de l'arbre contient au moins une clé (sauf si l'arbre est vide).

Nous allons esquisser le fonctionnement de la suppression, au lieu de présenter le pseudo code. La figure 18.8 illustre les divers cas de la suppression de clé dans un B-arbre.

© Dunod – La photocopie non autorisée est un délit

Scanned with CamScanner

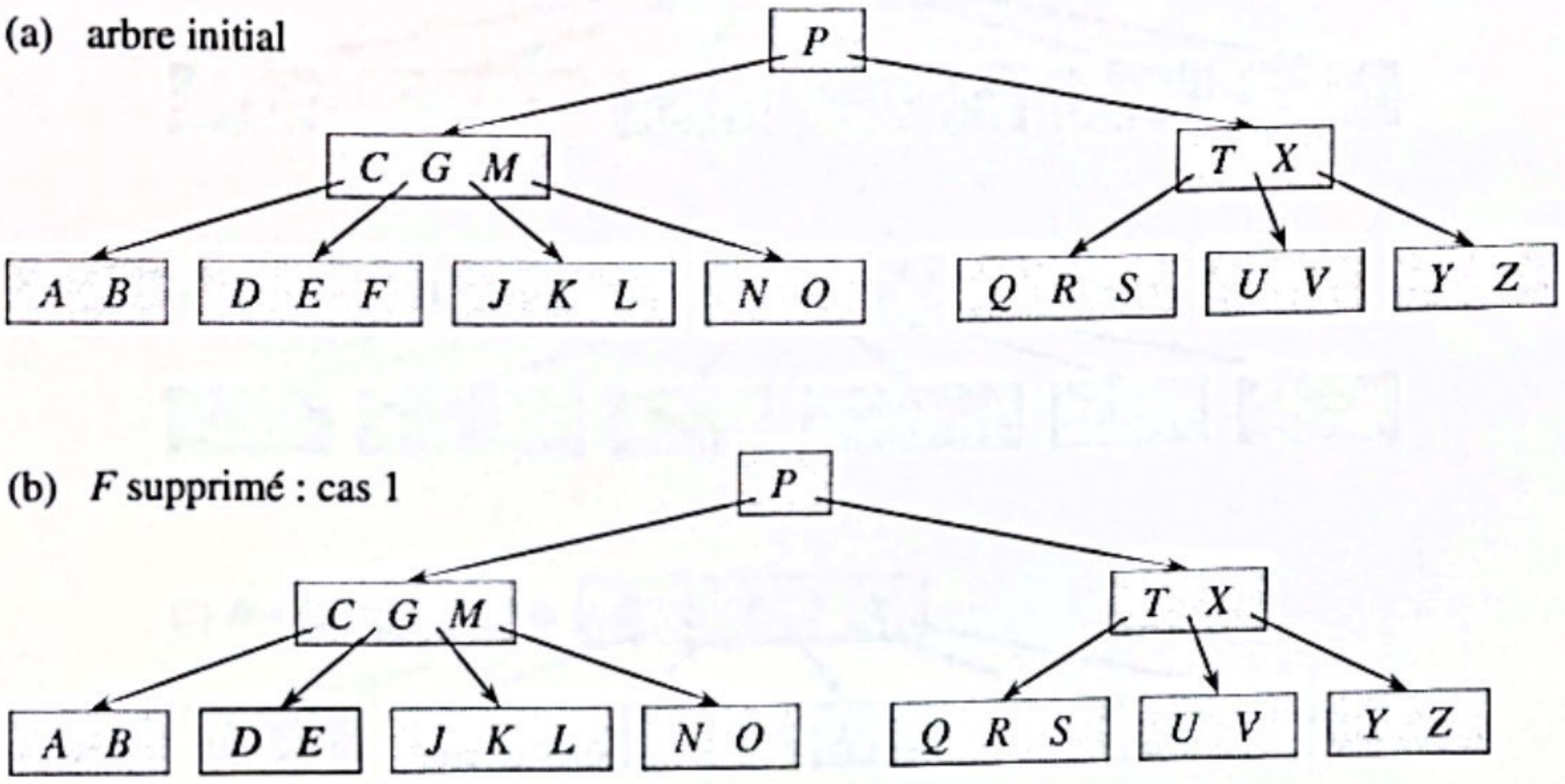
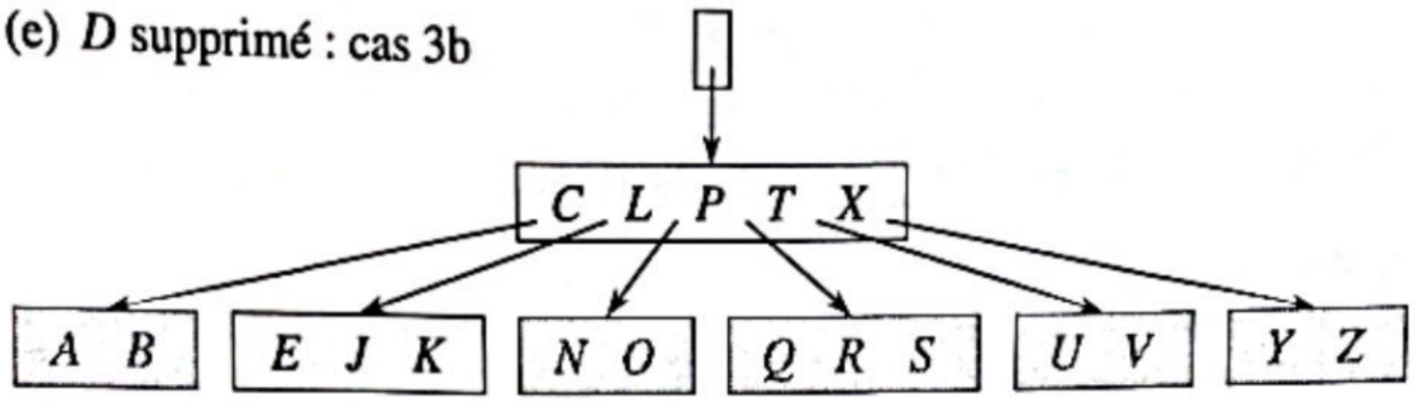


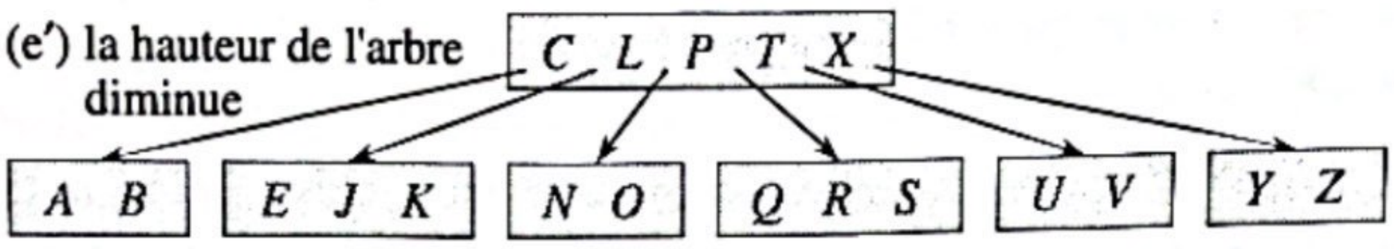
Figure 18.8 Suppression de clés dans un B-arbre. Le degré minimal de ce B-arbre est $t = 3$, de sorte qu'un nœud (autre que la racine) ne peut pas avoir moins de 2 clés. Les nœuds qui sont modifiés sont en gris clair. **(a)** Le B-arbre de la figure 18.7(e). **(b)** Suppression de *F*. C'est le cas 1 : suppression simple dans une feuille. **(c)** Suppression de *M*. C'est le cas 2a : le prédécesseur *L* de *M* remonte pour prendre la place de *M*. **(d)** Suppression de *G*. C'est le cas 2c : *G* est poussé vers le bas pour créer le nœud *DEGJK*, puis *G* est supprimé de cette feuille (cas 1). **(e)** Suppression de *D*. C'est le cas 3b : la récursivité ne peut pas descendre sur le nœud *CL*, car il n'a que 2 clés ; donc, *P* est poussé vers le bas et fusionné avec *CL* et *TX* pour former *CLPTX* ; ensuite, *D* est supprimé d'une feuille (cas 1). **(e')** Après (d), la racine est supprimée et la hauteur de l'arbre diminue de un. **(f)** Suppression de *B*. C'est le cas 3a : *C* est déplacé pour occuper la place de *B* et *E* est déplacé pour occuper la place de *C*.

Scanned with CamScanner

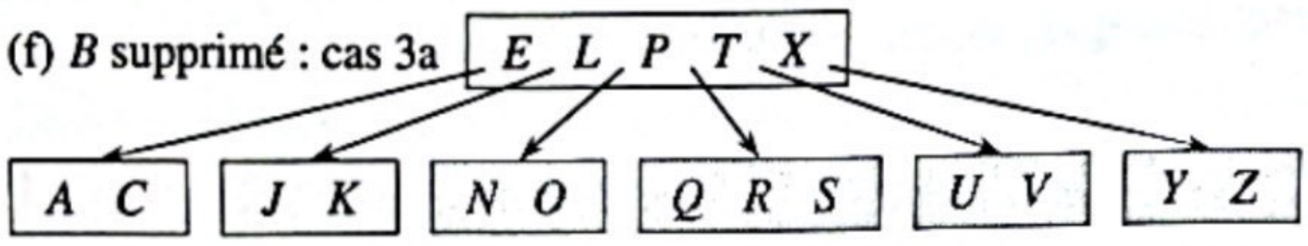
(e) *D* supprimé : cas 3b



(e′) la hauteur de l'arbre diminue



(f) *B* supprimé : cas 3a



1) Si la clé k est dans le nœud x et que x est une feuille, on supprime la clé k de x.
2) Si la clé k est dans le nœud x et que x est un nœud interne, on procède comme suit.
   a) Si l'enfant y qui précède k dans le nœud x a au moins t clés, on cherche le prédécesseur k' de k dans le sous-arbre enraciné en y. On supprime récursivement k', puis on remplace k par k' dans x. (Trouver k' et le supprimer sont deux actions qui peuvent se faire dans une même passe descendante.)
   b) De manière symétrique, si l'enfant z qui suit k dans le nœud x a au moins t clés, on cherche le successeur k' de k dans le sous-arbre enraciné en z. On supprime récursivement k', puis on remplace k par k' dans x. (Trouver k' et le supprimer sont deux actions qui peuvent se faire dans une même passe descendante.)
   c) Sinon, si y et z n'ont que $t - 1$ clés chacun, on fusionne k avec le contenu de z et on fait passer le tout dans y ; ce faisant, x perd k plus le pointeur vers z et y contient désormais $2t - 1$ clés. Ensuite, on libère z et l'on supprime récursivement k de y.
3) Si la clé k ne figure pas dans le nœud interne x, on cherche la racine $c_i[x]$ du sous-arbre contenant k (pour autant que k soit dans l'arbre). Si $c_i[x]$ n'a que $t - 1$ clés, on exécute l'étape 3a ou 3b (selon les besoins) pour garantir que l'on va descendre sur un nœud qui contient bien au moins t clés. Ensuite, on termine en appliquant la récursivité à l'enfant approprié de x.
   a) Si $c_i[x]$ n'a que $t - 1$ clés mais qu'il a un frère immédiat ayant au moins t clés, on donne à $c_i[x]$ une clé supplémentaire de la façon que voici : on prend une clé de x que l'on fait descendre dans $c_i[x]$, on prend une clé du frère immédiat (de gauche ou de droite) de $c_i[x]$ que l'on fait monter dans x, et on déplace le pointeur d'enfant approprié pour le faire passer du frère à $c_i[x]$.

© Dunod – La photocopie non autorisée est un délit

Scanned with CamScanner

b) Si les frères immédiats de $c_i[x]$ et de $c_i[x]$'s ont chacun $t - 1$ clés, on fusionne $c_i[x]$ avec un frère ; cela implique qu'il faut faire descendre une clé de x dans le nœud nouvellement fusionné pour qu'elle devienne la clé médiane de ce nœud.

Comme la plupart des clés d'un B-arbre sont dans les feuilles, on peut s'attendre à ce que, dans la pratique, les suppressions de clé se fassent le plus souvent sur des feuilles. La procédure SUPPRIMER-B-ARBRE travaille alors en une seule passe descendante, sans risque de devoir revenir en arrière dans l'arbre. En revanche, quand on supprime une clé d'un nœud interne, la procédure descend l'arbre mais elle peut éventuellement être amenée à revenir sur le nœud dont on a supprimé la clé, pour remplacer la clé par son prédécesseur ou successeur (cas 2a et 2b).

Cette procédure peut paraître compliquée, mais elle ne demande que $O(h)$ accès disque pour un B-arbre de hauteur h ; en effet, il n'y a que $O(1)$ appels LIRE-DISQUE et ÉCRIRE-DISQUE entre les invocations récursives de la procédure. Le temps CPU requis est $O(th) = O(t \log_t n)$.

Exercices

18.3.1 Montrer le résultat de la suppression dans l'ordre de C, P et V dans l'arbre de la figure 18.8(f).

18.3.2 Écrire le pseudo code de SUPPRIMER-B-ARBRE.

PROBLÈMES

18.1. Piles sur un espace de stockage secondaire

On considère l'implémentation d'une pile dans un ordinateur assez limité en mémoire principale et bien pourvu en espace disque. Les opérations EMPILER et DÉPILER sont supportées pour des valeurs contenues dans un mot. La pile devra pouvoir croître jusqu'à dépasser largement la mémoire principale disponible, ce qui impose d'en stocker la plus grande partie sur disque.

Conserver la pile entièrement sur le disque est simple quoique peu efficace. On garde en mémoire un pointeur vers la pile, qui représente l'adresse sur disque du sommet de la pile. Si le pointeur a une valeur p, l'élément du sommet est le (p mod m)ème mot sur la page $\lfloor p/m \rfloor$ du disque, où m est le nombre de mots par page.

Pour implémenter l'opération EMPILER, on incrémente le pointeur de pile, on charge en mémoire la page disque concernée, on copie l'élément à empiler dans le mot approprié sur la page et on écrit la page ainsi modifiée sur le disque. L'opération DÉPILER est similaire. On décrémente le pointeur de pile, on charge la page concernée à partir du disque et on retourne le sommet de la pile. Il est inutile de réécrire la page sur le disque puisqu'elle n'est pas modifiée par l'opération.

Scanned with CamScanner

Les opérations de disque étant relativement coûteuses, on compte deux choses pour toute implémentation : le nombre total d'accès disque et le temps processeur total. Un accès disque à une page de m mots compte pour un accès disque et pour un temps CPU de $\Theta(m)$.

a. Asymptotiquement, quel est le nombre d'accès au disque dans le cas le plus défavorable pour n opérations de pile ayant recours à cette implémentation naïve ? Quel est le temps CPU pris par les n opérations de pile ? (Exprimer les résultats en fonction de m et n. Idem pour les questions suivantes.)

On considère à présent une implémentation pour laquelle une page de la pile est conservée en mémoire. (On réserve également une petite quantité de mémoire pour savoir quelle page se trouve en mémoire à un instant donné.) Une opération de pile ne peut s'effectuer que si la page disque concernée réside en mémoire. Si besoin est, la page se trouvant en mémoire peut être sauvegardée sur le disque et une nouvelle page peut être chargée à partir du disque. Si la page disque se trouve déjà en mémoire, aucun accès au disque n'est nécessaire.

b. Quel est, dans le cas le plus défavorable, le nombre d'accès au disque requis pour n opérations EMPILER ? Quel est le temps CPU requis ?

c. Quel est, dans le cas le plus défavorable, le nombre d'accès au disque requis pour n opérations de pile ? Quel est le temps CPU requis ?

Supposons maintenant que la pile soit implémentée en conservant deux pages en mémoire (en plus d'un petit nombre de mots pour la gestion interne).

d. Décrire comment gérer les pages de pile pour que le nombre amorti d'accès au disque pour toute opération de pile soit $O(1/m)$ et le temps CPU amorti pour toute opération de pile soit $O(1)$.

18.2. Jointure et scission d'arbres 2-3-4

L'opération ***jointure*** prend deux ensembles dynamiques S' et S'' et un élément x tel que, pour tout $x' \in S'$ et $x'' \in S''$, on ait $clé[x'] < clé[x] < clé[x'']$. Elle retourne un ensemble $S = S' \cup \{x\} \cup S''$. L'opération ***scission*** est un peu « l'inverse » de la jointure : étant donné un ensemble dynamique S et un élément $x \in S$, elle crée un ensemble S' constitué de tous les éléments de $S - \{x\}$ dont les clés sont inférieures à $clé[x]$ et un ensemble S'' constitué de tous les éléments de $S - \{x\}$ dont les clés sont supérieures à $clé[x]$. Dans ce problème, on cherche à savoir comment implémenter ces opérations sur des arbres 2-3-4. On suppose par commodité que les éléments ne contiennent que des clés et que toutes les valeurs de clé sont distinctes.

a. Montrer comment gérer, pour tout nœud x d'un arbre 2-3-4, la hauteur du sous-arbre enraciné en x dans un champ $hauteur[x]$. S'assurer que l'implémentation retenue n'affecte pas les temps d'exécution asymptotiques de la recherche, de l'insertion et de la suppression.

© Dunod – La photocopie non autorisée est un délit

Scanned with CamScanner

b. Montrer comment implémenter l'opération de jointure. Étant donnés deux arbres 2-3-4 T' et T'' et une clé k, la jointure devra s'exécuter en $O(1 + |h' - h''|)$, où h' et h'' sont les hauteurs respectives de T' et T''.

c. Soit T un arbre 2-3-4. Soient p le chemin reliant la racine de T à une clé donnée k, S' l'ensemble des clés de T inférieures à k et S'' l'ensemble des clés de T supérieures à k. Montrer que p partage S' en un ensemble d'arbres $\{T'_0, T'_1, \ldots, T'_m\}$ et un ensemble de clés $\{k'_1, k'_2, \ldots, k'_m\}$, où, pour $i = 1, 2, \ldots, m$, on a $y < k'_i < z$ pour toutes clés $y \in T'_{i-1}$ et $z \in T'_i$. Quelle est la relation entre les hauteurs de T'_{i-1} et T'_i ? Décrire la façon dont p partage S'' en deux ensembles d'arbres et de clés.

d. Montrer comment implémenter l'opération de scission sur T. Utiliser l'opération de jointure pour rassembler les clés de S' dans un même arbre 2-3-4 T' et les clés de S'' dans un même arbre 2-3-4 T''. Le temps d'exécution de l'opération de scission devra être $O(\lg n)$, où n est le nombre de clés de T. (*Conseil :* Les coûts des jointures doivent s'annuler mutuellement en partie.)

NOTES

Knuth [185], Aho, Hopcroft et Ullman [5], et Sedgewick [269] fournissent des études plus détaillées sur les arbres équilibrés et les B-arbres. Comer [66] couvre de façon exhaustive les B-arbres. Guibas et Sedgewick [135] étudient les relations entre les différentes sortes d'arbres équilibrés, notamment les arbres rouge-noir et les arbres 2-3-4.

En 1970, J. E. Hopcroft inventa les arbres 2-3, précurseurs des B-arbres et des arbres 2-3-4, dans lesquels tout nœud interne a deux ou trois enfant. Les B-arbres ont été introduits par Bayer et McCreight en 1972 [32] ; ils n'expliquent pas le choix de ce nom.

Bender, Demaine et Farach-Colton [37] ont étudié les performances de B-arbres dans un contexte de gestion hiérarchisée de la mémoire. Leurs algorithmes ***à transparence de cache*** donnent de bonnes performances sans se soucier explicitement de la taille des données qui sont transférées entre les divers niveaux de la hiérarchie des mémoires.

Scanned with CamScanner